Лабораторная работа №3

Моделирование графов атак

Аристова Арина Олеговна

Содержание

1 Цель работы **4**

2 Задание **5**

3 Выполнение лабораторной работы **6**

3.1 Ядро моделирования графа атак 6
3.2 Однократный запуск эксперимента 12
3.3 Визуализация и анализ графа 15
3.4 Исследование масштабируемости 20
3.5 Параметрическое исследование 24
3.6 Контрольные вопросы 28

4 Выводы **30**

Список литературы **31**

Список иллюстраций

3.1 Файл attack_graph.jl 6
3.2 Файл ag_run_experiment.jl 12
3.3 Результат выполнения ag_run_experiment.jl в консоли 15
3.4 Сохранённый файл - результат выполнения ag_run_experiment.jl в консоли 15
3.5 Файл ag_analyze.jl 16
3.6 Результат выполнения ag_analyze.jl в консоли 19
3.7 Полученный график - результат выполнения ag_analyze.jl 20
3.8 Файл ag_convergence.jl 21
3.9 Полученный график - результат выполнения convergence.jl 24
3.10 Сохранённый файл - результат выполнения convergence.jl в консоли 24
3.11 Файл parameter_sweep.jl 25
3.12 Полученный график - результат выполнения parameter_sweep.jl . 28
3.13 Сохранённый файл - результат выполнения parameter_sweep.jl в консоли 28

1 Цель работы

- Освоить методы построения и анализа графов атак для оценки уязвимостей сетевой инфраструктуры.
- На примере моделирования атак на корпоративную сеть изучить:
  - представление сетевой топологии и уязвимостей в виде ориентированного графа;
  - алгоритмы поиска всех возможных путей атаки от начальных точек до целевых активов;
  - расчёт метрик центральности для определения критических узлов;
  - визуализацию графа с цветовой индикацией уровня риска;
  - оценку вероятности успешной атаки с учётом сложности эксплуатации уязвимостей.

2 Задание

- Построить граф атак для заданной топологии сети.
- Реализовать алгоритм поиска всех путей от заданного источника к цели.
- Рассчитать метрики центральности для всех узлов и выявить наиболее критичные.
- Визуализировать граф, раскрашивая узлы в зависимости от степени риска
- Присвоить каждому ребру вес (вероятность успешной атаки) и вычислить наи- более вероятный путь атаки

3 Выполнение лабораторной работы

3.1 Ядро моделирования графа атак

Содержит все функции для построения ориентированного графа атак, поиска путей, расчёта метрик центральности, присвоения весов и нахождения наиболее вероятного пути. Это основной модуль, используемый всеми скриптами.

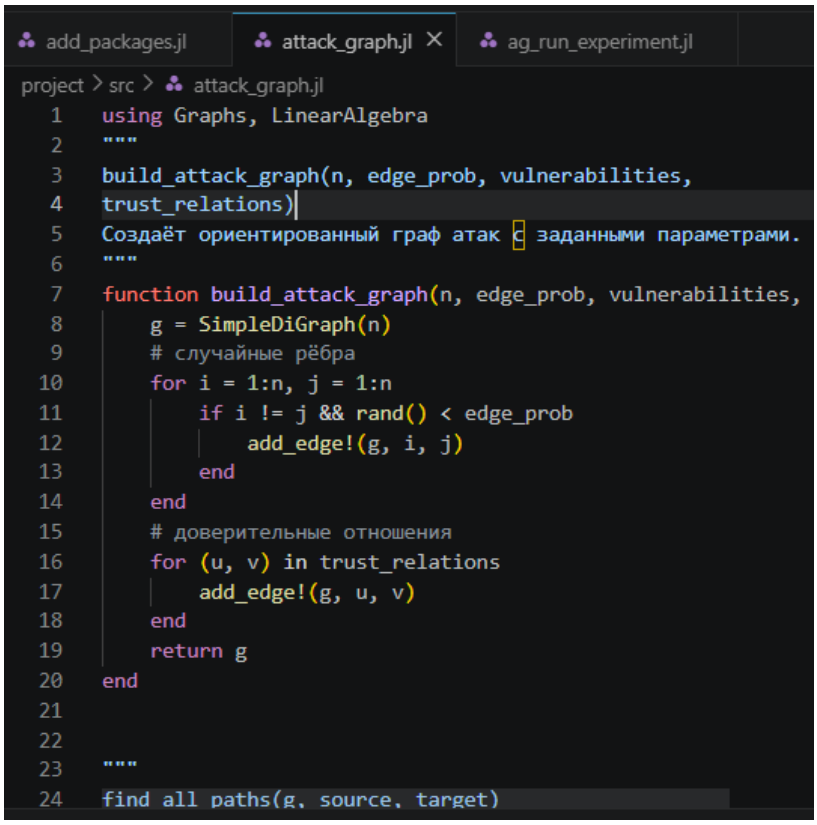

Рисунок 3.1: Файл attack_graph.jl

```julia
cd(joinpath(@__DIR__, "../project"))
using Graphs, LinearAlgebra
"""
build_attack_graph(n, edge_prob, vulnerabilities,
trust_relations)
Создаёт ориентированный граф атак с заданными параметрами.
"""
function build_attack_graph(n, edge_prob, vulnerabilities, trust_relations)
    g = SimpleDiGraph(n)
    for i = 1:n, j = 1:n
        if i != j && rand() < edge_prob
            add_edge!(g, i, j)
        end
    end
    for (u, v) in trust_relations
        add_edge!(g, u, v)
    end
    return g
end



"""
find_all_paths(g, source, target)
Рекурсивный поиск всех простых путей от source до target.
"""

function find_all_paths(g, source, target)
    paths = []
```

```
    function dfs(current, path)
        if current == target
            push!(paths, copy(path))
            return
        end
        for neighbor in outneighbors(g, current)
            if !(neighbor in path)
                push!(path, neighbor)
                dfs(neighbor, path)
                pop!(path)
            end
        end
    end
    dfs(source, [source])
    return paths
end



    """
    compute_centrality_metrics(g)
    Вычисляет основные метрики центральности.
    """
    function compute_centrality_metrics(g)
        indeg = indegree(g)
        outdeg = outdegree(g)
        betweenness = betweenness_centrality(g)
        closeness = closeness_centrality(g)
        pagerank = simple_pagerank(g) # используем нашу реализацию
```

```
        return Dict(
            :in_degree => indeg,
            :out_degree => outdeg,
            :betweenness => betweenness,
            :closeness => closeness,
            :pagerank => pagerank,
        )
    end
    """
    assign_edge_weights(g, cvss_scores)
    Присваивает каждому ребру вес на основе CVSS-оценок (по умолчанию
    0.5).
    """
    function assign_edge_weights(g, cvss_scores)
        weights = Dict{Edge,Float64}()
        for e in edges(g)
            u, v = src(e), dst(e)
            key = (u, v)
            weight = get(cvss_scores, key, 0.5)
            weights[e] = weight
        end
    return weights
end
"""
most_likely_path(g, source, target, weights)
Находит путь с максимальным произведением вероятностей.
"""
function most_likely_path(g, source, target, weights)
```

```
    n = nv(g)
    distmx = fill(Inf, n, n)
    for e in edges(g)
        u, v = src(e), dst(e)
        w = weights[e]
        distmx[u, v] = -log(w)
    end
    state = dijkstra_shortest_paths(g, source, distmx)
    dist = state.dists
    parents = state.parents
    if dist[target] == Inf
        return [], 0.0
    end
    path = Int[]
    current = target
    while current != source
        push!(path, current)
        current = parents[current]
    end
    push!(path, source)
    reverse!(path)
    prob = exp(-dist[target])
    return path, prob
end

"""
simple_pagerank(g; □=0.85, max_iter=100, tol=1e-6)
Простая реализация PageRank для ориентированного графа.
```

```julia
"""
function simple_pagerank(g; □ = 0.85, max_iter = 100, tol = 1e-6)
    n = nv(g)
    n == 0 && return Float64[]
    pr = fill(1.0 / n, n)
    for _ = 1:max_iter
        pr_new = fill((1-□)/n, n)
        for i = 1:n
            outdeg = outdegree(g, i)
            if outdeg > 0
                for j in outneighbors(g, i)
                    pr_new[j] += □ * pr[i] / outdeg
                end
            else
                for j = 1:n
                    pr_new[j] += □ * pr[i] / n
                end
            end
        end
        diff = maximum(abs.(pr_new - pr))
        pr = pr_new
        if diff < tol
            break
        end
    end
    return pr
end
```

```
simple_pagerank
```

3.2 Однократный запуск эксперимента

Файл scripts/ag_run_experiment.jl выполняет построение графа атак для заданных параметров, находит все пути, вычисляет метрики центральности, определяет наиболее вероятный путь и со- храняет результаты в JLD2-файл. Предназначен для генерации данных, которые будут анализироваться в других скриптах

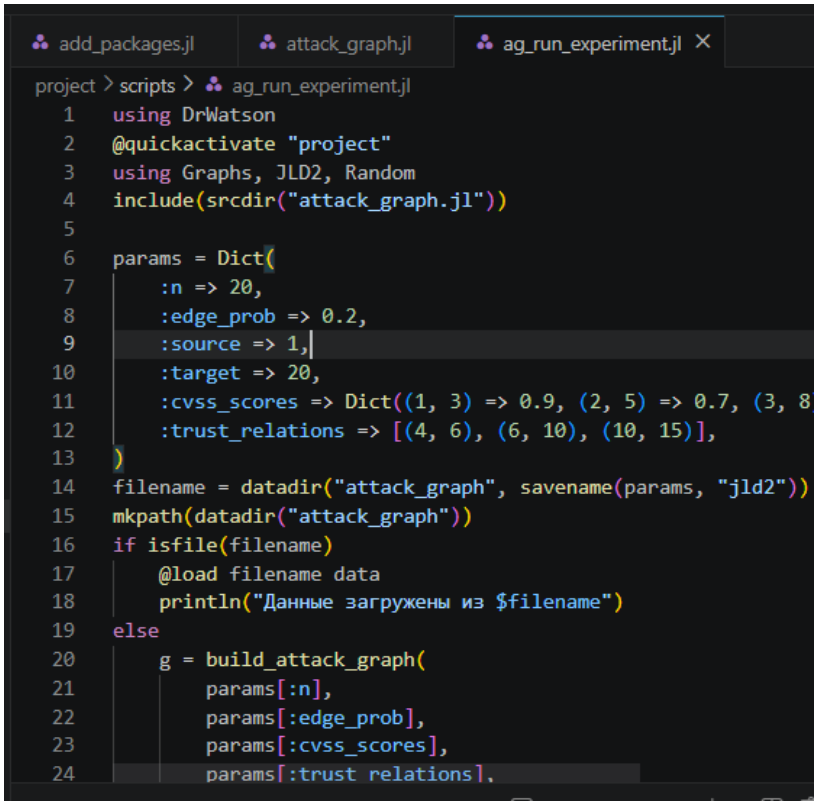

Рисунок 3.2: Файл ag_run_experiment.jl

```
cd(joinpath(@__DIR__, "../project"))
using DrWatson
@quickactivate "project"
using Graphs, JLD2, Random
include(srcdir("attack_graph.jl"))
```

```
params = Dict(
    :n => 20,
    :edge_prob => 0.2,
    :source => 1,
    :target => 20,
    :cvss_scores => Dict((1, 3) => 0.9, (2, 5) => 0.7, (3, 8) => 0.8, (5, 20) => 0.95
    :trust_relations => [(4, 6), (6, 10), (10, 15)],
)
filename = datadir("attack_graph", savename(params, "jld2"))
mkpath(datadir("attack_graph"))
if isfile(filename)
    @load filename data
    println("Данные загружены из $filename")
else
    g = build_attack_graph(
        params[:n],
        params[:edge_prob],
        params[:cvss_scores],
        params[:trust_relations],
    )
    paths = find_all_paths(g, params[:source], params[:target])
    metrics = compute_centrality_metrics(g)
    weights = assign_edge_weights(g, params[:cvss_scores])
    likely_path, prob = most_likely_path(g, params[:source],
    params[:target], weights)
    data = Dict(
        :graph => g,
        :paths => paths,
```

```
        :metrics => metrics,
        :weights => weights,
        :likely_path => likely_path,
        :probability => prob,
    )
    @save filename data params
    println("Результаты сохранены в $filename")
end

println("Количество путей атаки: ", length(data[:paths]))
println("Наиболее вероятный путь: ", data[:likely_path])
println("Вероятность успеха: ", data[:probability])
```

```
Данные загружены из C:\Users\arist\Github\2026-1--study--security-
modeling\labs\lab03\project\data\attack_graph\edge_prob=0.2_n=20_source=1_target=20.j
Количество путей атаки: 1
Наиболее вероятный путь: [1, 20]
Вероятность успеха: 0.5
```

Результат:

В папке data/attack_graph/ создаётся файл. В нём хранятся:

- :graph - объект графа;
- :paths - массив всех путей;
- :metrics - словарь с метриками центральности;
- :weights - веса рёбер;
- :likely_path - массив узлов наиболее вероятного пути;
- :probability - числовая вероятность успеха

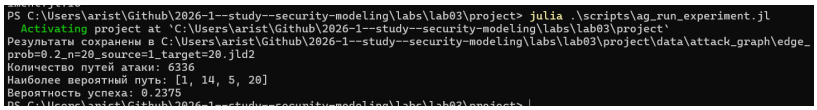

Рисунок 3.3: Результат выполнения ag_run_experiment.jl в консоли

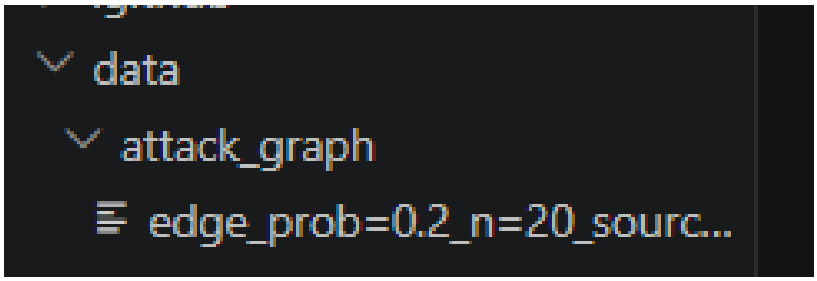

Рисунок 3.4: Сохранённый файл - результат выполнения ag_run_experiment.jl в консоли

3.3 Визуализация и анализ графа

Файл scripts/ag_analyze.jl загружает сохранённые данные. Строит наглядное изображение графа (с цвето- вой индикацией по PageRank). Выводит статистику: количество узлов, рёбер, пути, топ-5 критичных узлов по in-degree и PageRank.

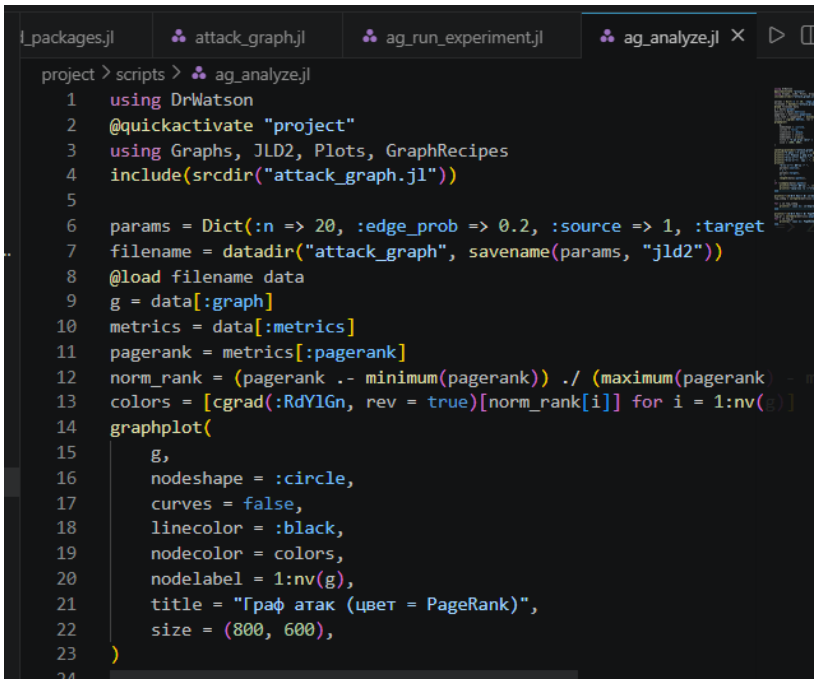

Рисунок 3.5: Файл ag_analyze.jl

```
cd(joinpath(@__DIR__, "../project"))
using DrWatson
@quickactivate "project"
using Graphs, JLD2, Plots, GraphRecipes
include(srcdir("attack_graph.jl"))

params = Dict(:n => 20, :edge_prob => 0.2, :source => 1, :target => 20)
filename = datadir("attack_graph", savename(params, "jld2"))
@load filename data
g = data[:graph]
metrics = data[:metrics]
pagerank = metrics[:pagerank]
norm_rank = (pagerank .- minimum(pagerank)) ./ (maximum(pagerank) - minimum(pagerank)
colors = [cgrad(:RdYlGn, rev = true)[norm_rank[i]] for i = 1:nv(g)]
```

```julia
graphplot(
    g,
    nodeshape = :circle,
    curves = false,
    linecolor = :black,
    nodecolor = colors,
    nodelabel = 1:nv(g),
    title = "Граф атак (цвет = PageRank)",
    size = (800, 600),
)

savefig(plotsdir("attack_graph.png"))
println("График сохранён в ", plotsdir("attack_graph.png"))
println("=== Анализ графа атак ===")
println("Количество узлов: ", nv(g))
println("Количество рёбер: ", ne(g))
println(
    "Количество путей от ",
    params[:source],
    " к ",
    params[:target],
    ": ",
    length(data[:paths]),
)
if !isempty(data[:paths])
    println("Длины путей: ", [length(p) for p in data[:paths]])
    println("Наиболее вероятный путь: ", data[:likely_path])
end
```

```julia
println("\nТоп-5 узлов по in-degree:")
top_indeg = sortperm(metrics[:in_degree], rev = true)[1:5]

for i in top_indeg
    println(" Узел $i: in-degree = $(metrics[:in_degree][i])")
end

println("\nТоп-5 узлов по PageRank:")
top_pr = sortperm(metrics[:pagerank], rev = true)[1:5]
for i in top_pr
    println(" Узел $i: PageRank = $(round(metrics[:pagerank][i], digits=4))")
end
```

```
График сохранён в C:\Users\arist\Github\2026-1--study--security-
modeling\labs\lab03\project\plots\attack_graph.png
=== Анализ графа атак ===
Количество узлов: 20
Количество рёбер: 69
Количество путей от 1 к 20: 1
Длины путей: [2]
Наиболее вероятный путь: [1, 20]

Топ-5 узлов по in-degree:
 Узел 7: in-degree = 7
 Узел 16: in-degree = 7
 Узел 8: in-degree = 6
 Узел 10: in-degree = 5
 Узел 13: in-degree = 5
```

```
Топ-5 узлов по PageRank:
 Узел 7: PageRank = 0.1291
 Узел 10: PageRank = 0.0914
 Узел 8: PageRank = 0.0866
 Узел 16: PageRank = 0.0849
 Узел 15: PageRank = 0.0643
```

Результат:

- PNG-файл с визуализацией графа в папке plots/.
- Текстовый отчёт в консоли.

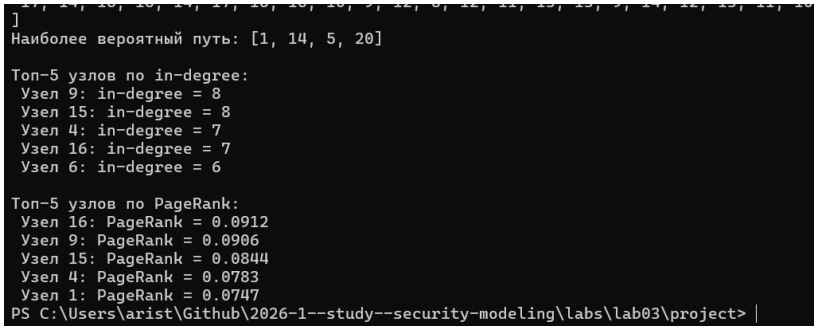

Рисунок 3.6: Результат выполнения ag_analyze.jl в консоли

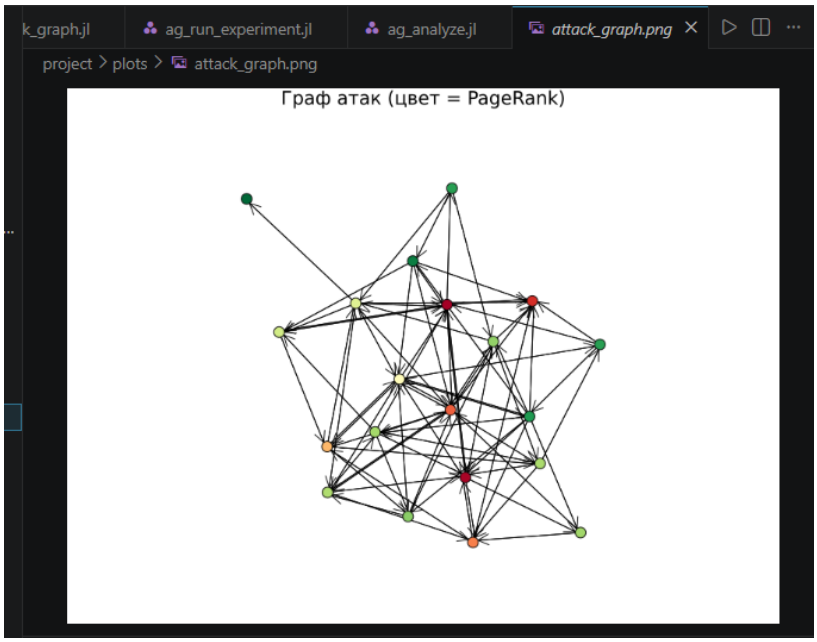

Рисунок 3.7: Полученный график - результат выполнения ag_analyze.jl

3.4 Исследование масштабируемости

Файл scripts/ag_convergence.jl исследование, как размер сети (число узлов) влияет на время поиска всех путей и на количество найденных путей. Позволяет оценить вычислительную сложность алгоритма.

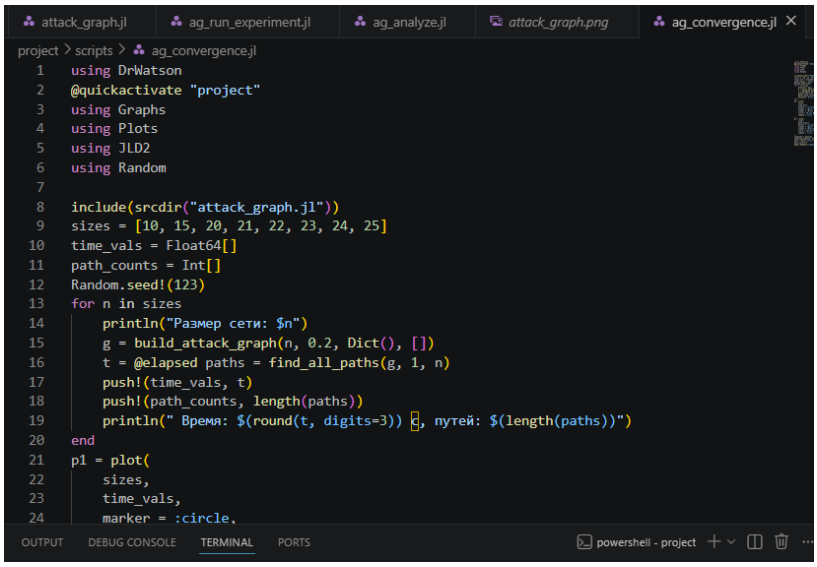

Рисунок 3.8: Файл ag_convergence.jl

```
cd(joinpath(@__DIR__, "../project"))
using DrWatson
@quickactivate "project"
using Graphs
using Plots
using JLD2
using Random

include(srcdir("attack_graph.jl"))
sizes = [10, 15, 20, 21, 22, 23, 24, 25]
time_vals = Float64[]
path_counts = Int[]
Random.seed!(123)
for n in sizes
    println("Размер сети: $n")
    g = build_attack_graph(n, 0.2, Dict(), [])
```

```
    t = @elapsed paths = find_all_paths(g, 1, n)
    push!(time_vals, t)
    push!(path_counts, length(paths))
    println(" Время: $(round(t, digits=3)) с, путей: $(length(paths))")
end
p1 = plot(
    sizes,
    time_vals,
    marker = :circle,
    xlabel = "Число узлов",
    ylabel = "Время (с)",
    title = "Время поиска всех путей",
    legend = false,
)
p2 = plot(
    sizes,
    path_counts,
    marker = :circle,
    xlabel = "Число узлов",
    ylabel = "Количество путей",
    title = "Количество путей атаки",
    legend = false,
)
combined = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(combined, plotsdir("convergence.png"))
data = Dict(:sizes => sizes, :times => time_vals, :path_counts => path_counts)
mkpath(datadir("convergence"))
@save datadir("convergence", "convergence_data.jld2") data
```

```
Размер сети: 10
 Время: 0.024 с, путей: 1
Размер сети: 15
 Время: 0.0 с, путей: 842
Размер сети: 20
 Время: 0.001 с, путей: 817
Размер сети: 21
 Время: 0.019 с, путей: 58966
Размер сети: 22
 Время: 0.228 с, путей: 355669
Размер сети: 23
 Время: 0.577 с, путей: 726135
Размер сети: 24
 Время: 8.922 с, путей: 10596200
Размер сети: 25
 Время: 11.275 с, путей: 19097246
```

Результат:

- График convergence.png, показывающий рост времени и числа путей с увели- чением сети (рис. 3.2).
- JLD2-файл с данными для возможного повторного использования.

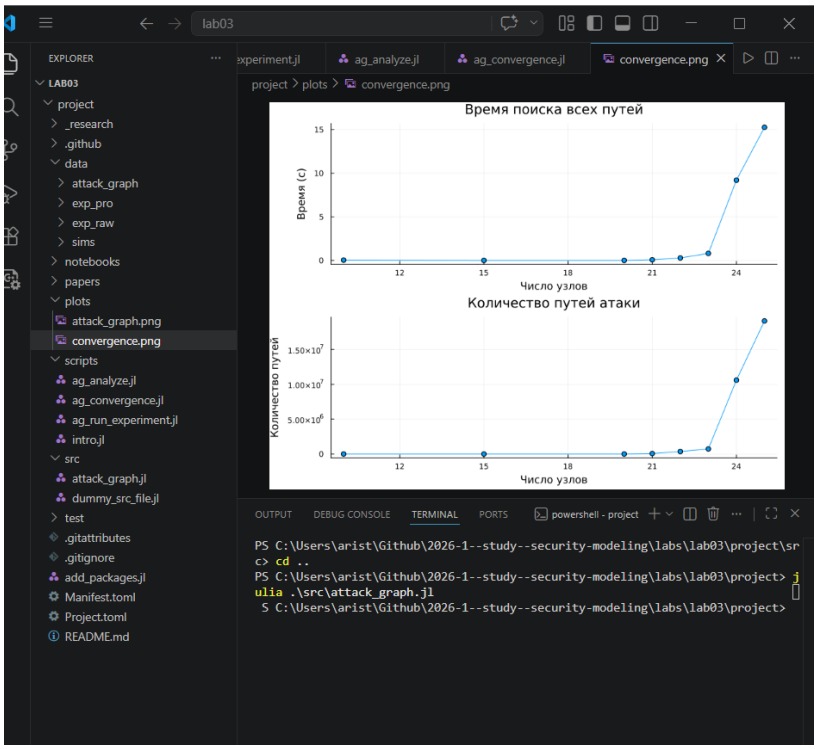

Рисунок 3.9: Полученный график - результат выполнения convergence.jl

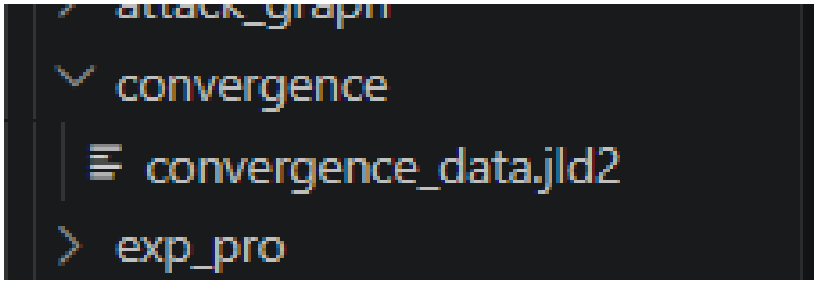

Рисунок 3.10: Сохранённый файл - результат выполнения convergence.jl в консоли

3.5 Параметрическое исследование

Файл scripts/parameter_sweep.jl изучает, как изменение плотности рёбер (вероятности случайного ребра) влияет на количество путей атаки, максимальную входящую степень и среднюю длину пути.

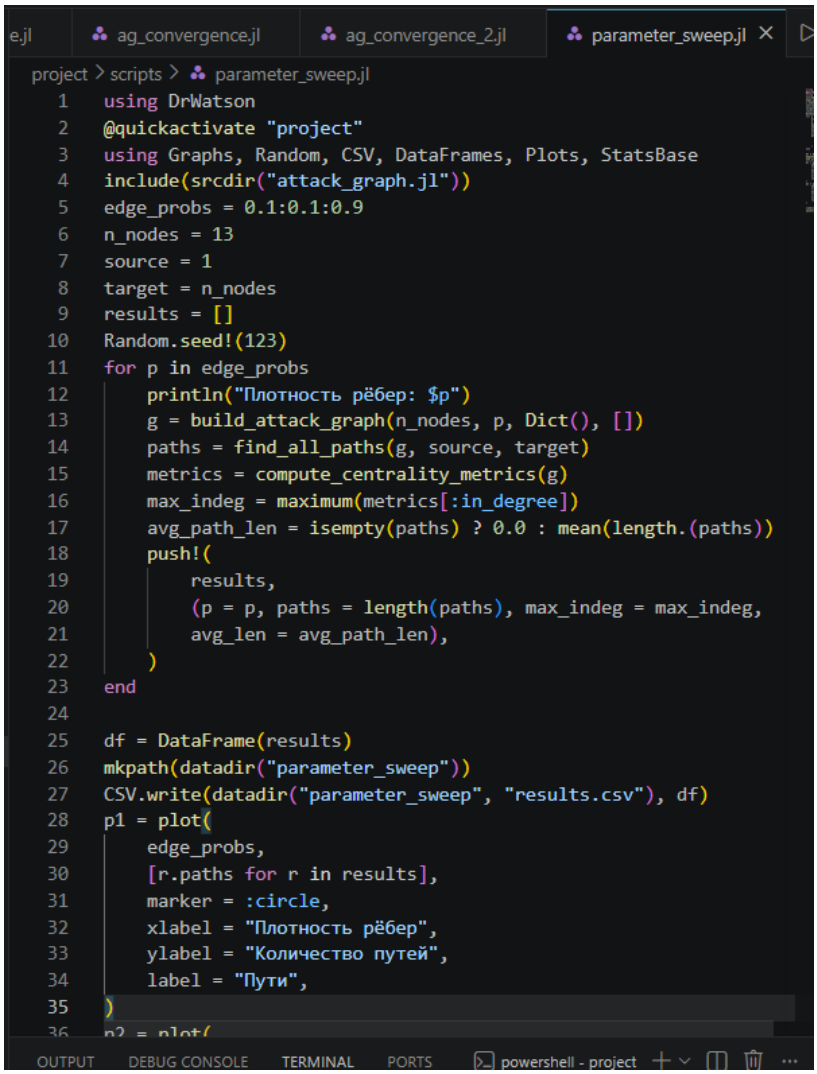

Рисунок 3.11: Файл parameter_sweep.jl

```
cd(joinpath(@__DIR__, "../project"))
using DrWatson
@quickactivate "project"
using Graphs, Random, CSV, DataFrames, Plots, StatsBase
include(srcdir("attack_graph.jl"))
edge_probs = 0.1:0.1:0.9
n_nodes = 13
source = 1
```

```
target = n_nodes
results = []
Random.seed!(123)
for p in edge_probs
    println("Плотность рёбер: $p")
    g = build_attack_graph(n_nodes, p, Dict(), [])
    paths = find_all_paths(g, source, target)
    metrics = compute_centrality_metrics(g)
    max_indeg = maximum(metrics[:in_degree])
    avg_path_len = isempty(paths) ? 0.0 : mean(length.(paths))
    push!(
        results,
        (p = p, paths = length(paths), max_indeg = max_indeg,
        avg_len = avg_path_len),
    )
end

df = DataFrame(results)
mkpath(datadir("parameter_sweep"))
CSV.write(datadir("parameter_sweep", "results.csv"), df)
p1 = plot(
    edge_probs,
    [r.paths for r in results],
    marker = :circle,
    xlabel = "Плотность рёбер",
    ylabel = "Количество путей",
    label = "Пути",
)
```

```
p2 = plot(
    edge_probs,
    [r.avg_len for r in results],
    marker = :circle,
    xlabel = "Плотность рёбер",
    ylabel = "Средняя длина пути",
    label = "Средняя длина",
)

combined = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(combined, plotsdir("parameter_sweep.png"))
```

```
Плотность рёбер: 0.1
Плотность рёбер: 0.2
Плотность рёбер: 0.3
Плотность рёбер: 0.4
Плотность рёбер: 0.5
Плотность рёбер: 0.6
Плотность рёбер: 0.7
Плотность рёбер: 0.8
Плотность рёбер: 0.9

"C:\\Users\\arist\\Github\\2026-1--study--security-modeling\\labs\\lab03\\project\\pl
```

Результат:

- Таблица результатов в CSV.
- Графики (рис. 3.3), показывающие, как увеличение связности графа (плотно- сти) приводит к росту числа путей и их средней длины.

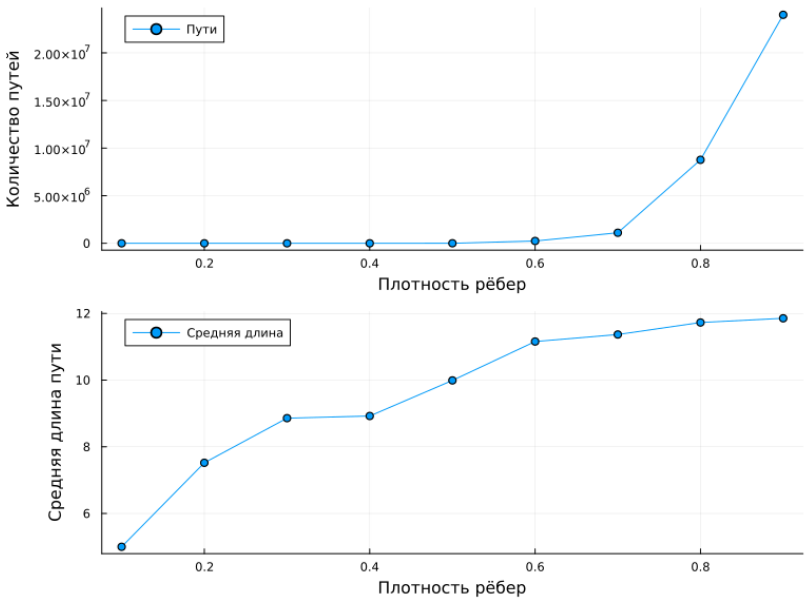

Рисунок 3.12: Полученный график - результат выполнения parameter_sweep.jl

Рисунок 3.13: Сохранённый файл - результат выполнения parameter_sweep.jl в консоли

3.6 Контрольные вопросы

1. Граф атак - направленный граф возможных действий злоумышленника для достижения цели. Вершины - состояния системы, рёбра - переходы через уязвимости. Используется для оценки защищённости, выявления маршрутов атак, расчёта метрик опасности, анализа рисков и расследования инцидентов.

2. Для поиска путей в графах атак применяются различные алгоритмы, например: BFS (кратчайший путь), Дейкстра (взвешенные графы), A* (с

эвристикой), Беллман–Форд (отрицательные веса), Флойд–Уоршелл (все пары вершин). Веса рёбер могут отражать сложность или вероятность успеха атаки.

3. Метрики центральности определяют важность узлов в графе.

- In-degree - число входящих рёбер (указывает на частоту атак на узел).
- PageRank - учитывает важность связанных узлов (показывает влияние на распространение угроз).

4. Вероятность успешной атаки оценивается через произведение весов рёбер на пути (вероятности успеха перехода). Для поиска наиболее вероятного пути используют модифицированный алгоритм Дейкстры или A*. Сложные модели - байесовские графы атак.

5. Ограничения модели: масштабируемость (экспоненциальный рост числа путей), неопределённость данных, динамичность сетей, сложность с циклами и защитными механизмами, высокие вычислительные затраты.

6. Учёт защитных механизмов (например, МЭ): добавление узлов/рёбер защиты, условные переходы (блокировка → обнуление веса), атрибуты правил фильтрации, вероятностные модели (шанс обнаружения), интеграция конфигураций правил доступа и многоуровневые модели безопасности.

4 Выводы

В результате данной лабораторно работы освоены методы построения и анализа графов атак для оценки уязвимостей сетевой инфраструктуры, а также выполнен пример моделирования атак на корпоративную сеть.

Список литературы

1. Описание лабораторных работ